

PROYECTO ACADÉMICO · INGENIERÍA DE SOFTWARE

AdoptaAPI

Sistema de Gestión de Adopción de Mascotas
API REST · ASP.NET Core MVC 8 · Azure

Institución / Curso

Desarrollo Web — Proyecto Lógico
Web → API

Metodología

Agile / Scrum

Stack tecnológico

ASP.NET Core MVC 8, C#,
JavaScript, SQL Server, Azure

Arquitectura

MVC en capas (Presentation ·
Business · Data Access)

Versión documento

1.0 — Agosto 2026

Integrantes

4 personas

Samanta Rojas · Killiam García · Jonnathan Rey · Santiago Cuervo

Índice de contenidos

1. Resumen ejecutivo
2. Contexto y objetivos
3. Alcance del proyecto
4. Épicas y módulos
5. Historias de usuario
6. Tareas y planificación
7. Modelo de datos
8. Diagrama Entidad Relación (Modelo de Datos)
9. Maquina de estados: Solicitud de Adopción
10. Diagrama de Casos de Uso
11. Diagrama de Clases
12. Diagrama de Secuencia
13. Diagrama de Componentes

1. Resumen ejecutivo

AdoptaAPI es una plataforma tecnológica orientada a centralizar y digitalizar el proceso de adopción de mascotas en Colombia. Conecta fundaciones de rescate animal con adoptantes potenciales mediante una **API REST documentada** y una interfaz web MVC, reemplazando la gestión fragmentada actual en redes sociales.

Aspecto	Descripción
Problema	Información dispersa, falta de trazabilidad y procesos informales en adopciones
Solución	API centralizada con filtros, flujo de solicitudes con estados y reportes de impacto
Usuarios	Fundaciones de rescate, adoptantes y visitantes
Entregables	API REST, Swagger UI, modelo relacional, diagramas UML, mockups Figma, despliegue Azure
Duración estimada	6 semanas (3 sprints + setup)

2. Contexto y objetivos

2.1 Objetivo general

Desarrollar una **API REST** que centralice y digitalice el proceso de **publicación, búsqueda y seguimiento de adopciones de mascotas**, conectando de forma ágil y trazable a fundaciones de rescate animal con potenciales adoptantes.

2.2 Objetivos específicos

ID	Objetivo	Indicador de éxito
OE-01	API REST documentada con Swagger	OpenAPI 3.0 publicado en /swagger
OE-02	Filtrado por especie, edad, tamaño y ciudad	GET /api/mascotas con query params funcionales
OE-03	Flujo de solicitud con estados trazables	5 estados con transiciones validadas
OE-04	Reportes para fundaciones	Dashboard con métricas disponibles vs. adoptadas
OE-05	Interfaz Swagger UI + mockups Figma	Prototipo navegable y API probada en vivo

2.3 Descripción del problema

La tenencia de mascotas en Colombia es un fenómeno significativo: cerca de 58% de los hogares colombianos tiene al menos una mascota, según cifras citadas por el director de Asuntos Económicos de Fenalco, con los perros como la especie preferida entre los hogares. Este volumen de tenencia va de la mano de una red de rescate igualmente amplia, pero profundamente informal: la mayoría de quienes rescatan animales en el país no son fundaciones con sede física, sino personas particulares que operan bajo el modelo de "hogar de paso" o *foster*, cuidando a los animales en sus propias casas mientras buscan adoptantes, un esquema muy dedicado pero con recursos limitados que constituye la mayor parte de la red de rescate en el país.

Esta atomización de actores —fundaciones legalmente constituidas, colectivos de rescatistas independientes y hogares de paso individuales— explica por qué la información sobre mascotas disponibles termina dispersa en decenas de perfiles distintos. La práctica común, tanto de fundaciones grandes como de rescatistas individuales, sigue siendo publicar en Instagram o Facebook, apoyándose en grupos como "Adopciones Bogotá Gatos" o "Felinos en Adopción Colombia" y en hashtags de campaña como #AdoptaEnBogota o #AdoptaNoCompres, además de jornadas presenciales realizadas en parques y centros comerciales en alianza con marcas o comercios. Si bien estas estrategias generan visibilidad puntual, no ofrecen ningún mecanismo de control centralizado ni de actualización automática del estado de cada animal.

A esto se suma un problema de confianza: dado que cualquier persona puede abrir un perfil o publicar en un grupo de Facebook ofreciendo mascotas en adopción, los propios adoptantes deben verificar manualmente que la fundación o el rescatista sea legítimo, evitando lugares que vendan mascotas de forma clandestina o sin garantías sanitarias. La ausencia de un registro único y verificable traslada esa carga de validación al usuario final, en lugar de resolverse a nivel de plataforma.

Por otro lado, el problema de fondo que busca atender la adopción —la sobrepoblación de animales en situación de calle— tiene una dimensión estructural: cada adopción efectiva libera un cupo físico en un refugio para que otro animal rescatado pueda recibir atención, lo que combate directamente la sobrepoblación y desincentiva la cría y venta indiscriminada de animales, una problemática que organizaciones como World Animal Protection buscan visibilizar a nivel global. Sin embargo, cuando el proceso de adopción es lento o poco trazable por depender de mensajes privados y comentarios dispersos, ese ciclo de rotación se hace más lento, y los refugios y hogares de paso permanecen saturados por más tiempo del necesario.

Finalmente, iniciativas existentes muestran que algunas fundaciones ya intentan paliar la falta de seguimiento de forma manual y no estandarizada —por ejemplo, mediante programas de "padrinazgo" en los que reportan periódicamente el estado del animal a quien realiza un aporte— como el "Plan Padrino" de Casa de Esperanza, que envía un reporte del estado del animal y del manejo del aporte, lo cual confirma que existe una necesidad reconocida de trazabilidad y reporte, pero resulta hoy con procesos artesanales, específicos de cada organización y no escalables.

En conjunto, estos antecedentes evidencian que el problema no es la falta de interés de fundaciones y adoptantes, sino la ausencia de una infraestructura tecnológica común que estandarice publicación, búsqueda, verificación y seguimiento del proceso de adopción —justificando la necesidad de una API centralizada como AdoptaAPI.

3. Alcance del proyecto

3.1 Dentro del alcance

- API REST: usuarios, mascotas, solicitudes, reportes
- Autenticación JWT por rol (fundación / adoptante)
- Filtros: especie, edad, tamaño, ciudad, estado
- Swagger UI interactivo
- Modelo relacional SQL Server + EF Core
- Interfaz web MVC (cliente de la API)
- Mockups navegables en Figma
- Despliegue en Azure App Service

3.2 Fuera del alcance

- Pasarela de pagos o donaciones
- Notificaciones push en tiempo real
- App móvil nativa
- Verificación legal/judicial de adopciones
- Verificación automatizada de fundaciones

4. Épicas y módulos

Épica	Módulo	Descripción	Prioridad	Sprint
E-01	Autenticación y Usuarios	Registro, login JWT, roles fundación/adoptante	Alta	Sprint 1
E-02	Gestión de Mascotas	CRUD fichas + filtros de búsqueda avanzada	Alta	Sprint 1
E-03	Solicitudes de Adopción	Flujo con máquina de estados trazable	Alta	Sprint 2
E-04	Reportes e Impacto	Métricas y dashboard para fundaciones	Media	Sprint 2
E-05	Documentación API	Swagger UI + OpenAPI 3.0	Media	Sprint 3
E-06	Interfaz Web y UX	Vistas MVC + mockups Figma navegables	Media	Sprint 1-3

5. Historias de usuario

HU-01 — Registro de fundación

Como fundación, quiero registrarme, para publicar mascotas.

- Campos: nombre, email, contraseña, ciudad, organización email único, contraseña hasheada, tipo Fundación, HTTP 201

HU-02 — Registro de adoptante

Como adoptante, quiero registrarme, para enviar solicitudes.

- Campos: nombre, email, contraseña, ciudad, tipo Adoptante, HTTP 201

HU-03 — Inicio de sesión

Como usuario, quiero iniciar sesión, para acceder según mi rol.

- JWT con claims userId, email, rol, HTTP 401 si inválido

HU-04 — Publicar mascota

Como fundación, quiero publicar una ficha, para que adoptantes la encuentren.

- Campos obligatorios: nombre, especie, edad, tamaño, ciudad, descripción, estado inicial Disponible, solo rol Fundacion

HU-05 — Buscar mascotas con filtros

Como visitante/adoptante, quiero filtrar mascotas, para encontrar la ideal.

- Filtros combinables: especie, edad, tamaño, ciudad, estado, paginación; endpoint público

HU-06 — Actualizar o eliminar mascota

Como fundación, quiero editar/eliminar mis fichas, para mantener información actualizada.

- Solo propietario; no eliminar con solicitudes En Revision/Aprobada

HU-07 — Ver detalle de mascota

Como adoptante, quiero ver info completa, para decidir adopción.

- Incluye datos de fundación; HTTP 404 si no existe

HU-08 — Enviar solicitud de adopción

Como adoptante, quiero solicitar adopción, para iniciar el proceso.

- Estado Pendiente; mascota Disponible; mascota pasa a EnProceso; HTTP 201

HU-09 — Gestionar solicitudes

Como fundación, quiero cambiar estados, para gestionar adopciones.

- Pendiente→EnRevision→Aprobada|Rechazada→Completada
- Al aprobar: mascota Adoptada; rechazar otras solicitudes activas

HU-10 — Consultar mis solicitudes

Como adoptante, quiero ver mis solicitudes, para hacer seguimiento.

- Solo del adoptante autenticado; orden por fecha descendente

HU-11 — Reporte de impacto

Como fundación, quiero ver reporte de mis mascotas, para medir impacto.

- Totales: disponibles, en proceso, adoptadas; filtro por fechas

HU-12 / HU-13 — Swagger UI y Mockups Figma

Como desarrollador / usuario, quiero consultar API y prototipos visuales.

- Swagger en /swagger con JWT; todos endpoints documentados con ejemplos.
Mockups Figma: flujo completo responsive (mobile, tablet, desktop).

6. Tareas y planificación

Sprint	IDs	Entregables clave	Duración
Sprint 0	T-01 a T-06	Solución .NET 8, EF Core, Swagger, Git, wireframes Figma	1 semana
Sprint 1	T-07 a T-15	Auth JWT, CRUD mascotas, filtros, vistas MVC, diagramas UML (casos de uso, clases)	2 semanas
Sprint 2	T-16 a T-24	Solicitudes + estados, reportes, panel fundación, diagramas (secuencia, componentes)	2 semanas
Sprint 3	T-25 a T-30	Swagger completo, pruebas, mockups finales, despliegue Azure, demo	1 semana

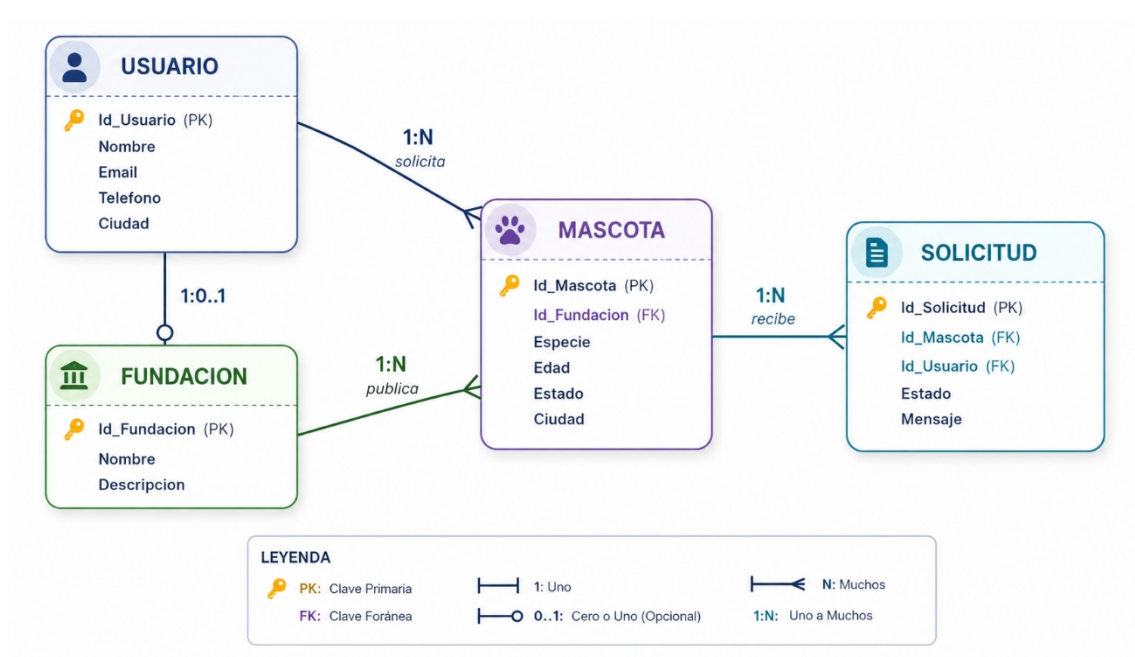
6.1 Detalle de tareas por sprint

ID	Tarea	Historia	Est.
T-07	AuthService + AuthController (JWT)	HU-01–03	8h
T-09	MascotaService + Controller (CRUD)	HU-04, HU-06	8h
T-10	Filtros y paginación GET /mascotas	HU-05	5h
T-16	SolicitudService + máquina de estados	HU-08, HU-09	10h
T-19	ReporteService + Controller	HU-11	5h
T-21	Panel fundación (solicitudes + reporte)	HU-09, HU-11	10h
T-29	Despliegue Azure App Service + SQL Azure	—	6h

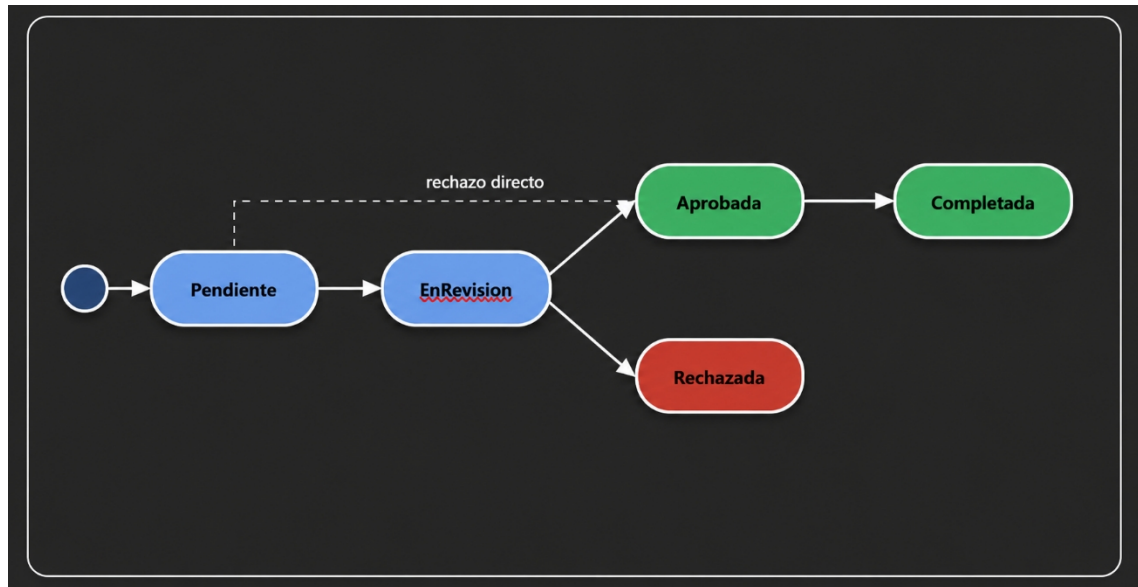
7. Modelo de datos

Entidad	Campos clave	Relaciones
USUARIO	Id_Usuario(Pk), Nombre, Email(Uk), Telefono, Ciudad	1:0..1 FUNDACION; 1:N SOLICITUD
FUNDACION	Id_Fundacion(Pk), Nombre, Descripcion	1:N MASCOTA
MASCOTA	Id_Mascota(Pk), Id_Fundacion(Fk), Especie, Edad, Ciudad, Estado	1:N SOLICITUD
SOLICITUD_ADOPCION	Id_Solicitud(Pk), Id_Mascota(Fk), Id_Usuario(Fk), Estado, Mensaje	N:1 MASCOTA, USUARIO

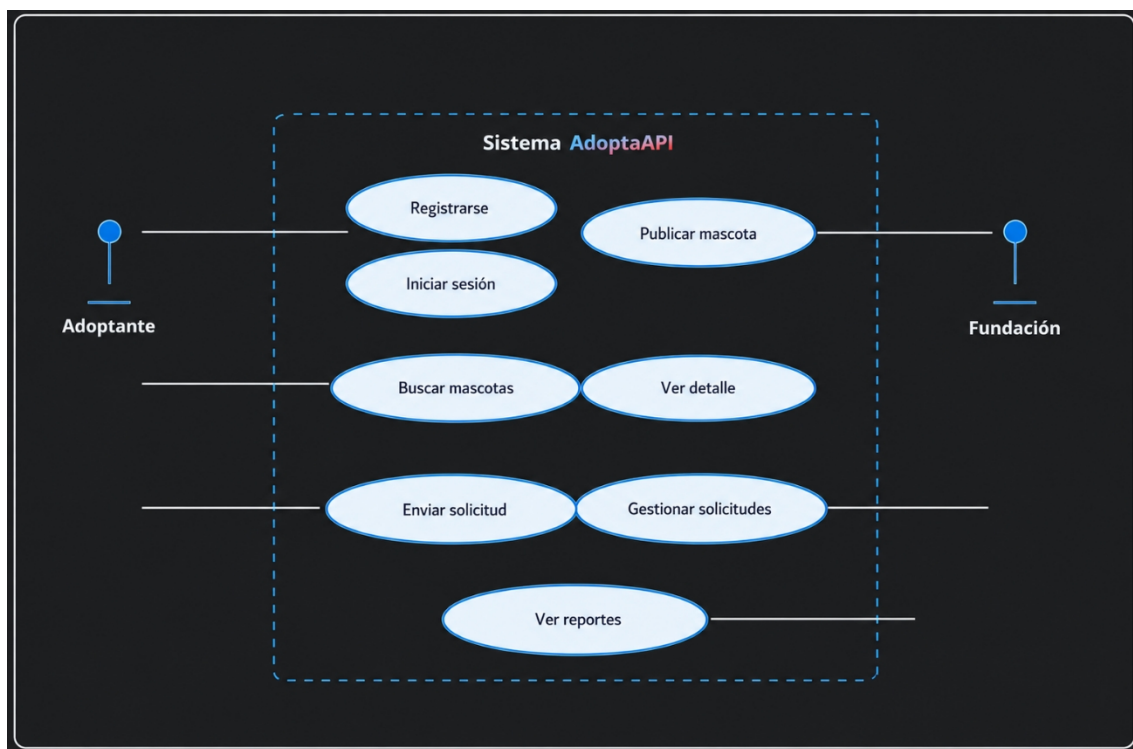
8. Diagrama Entidad Relación (Modelo de Datos)



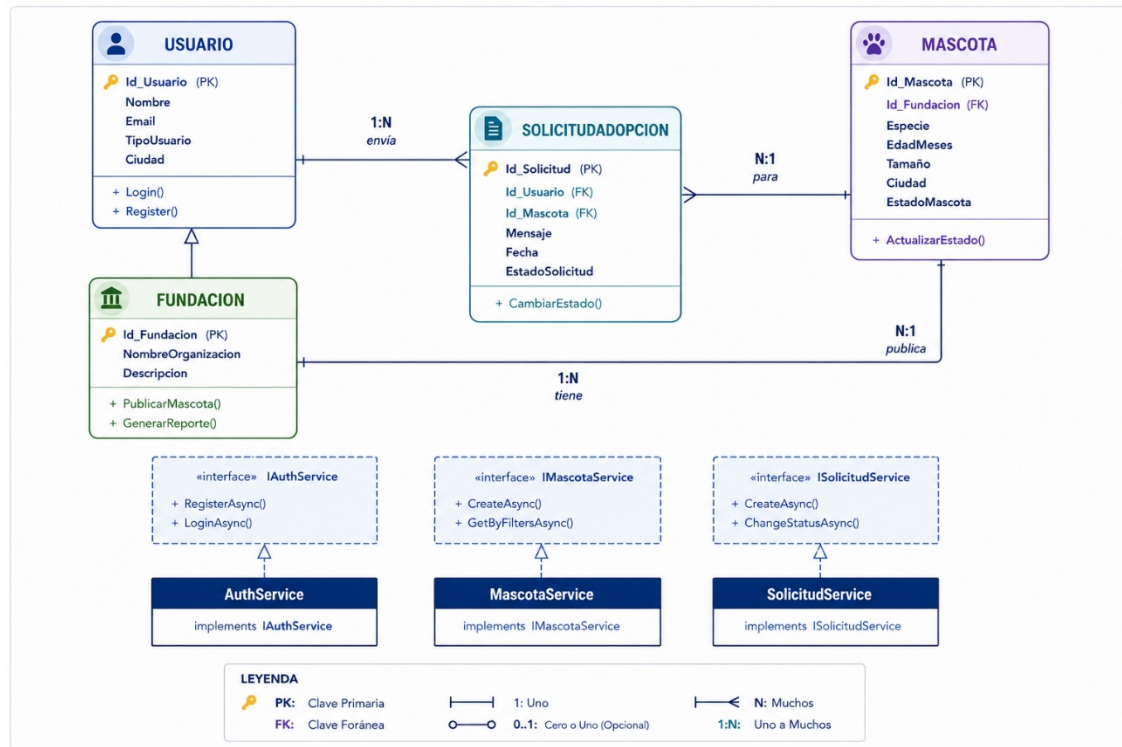
9. Máquina de estados: Solicitud de Adopción



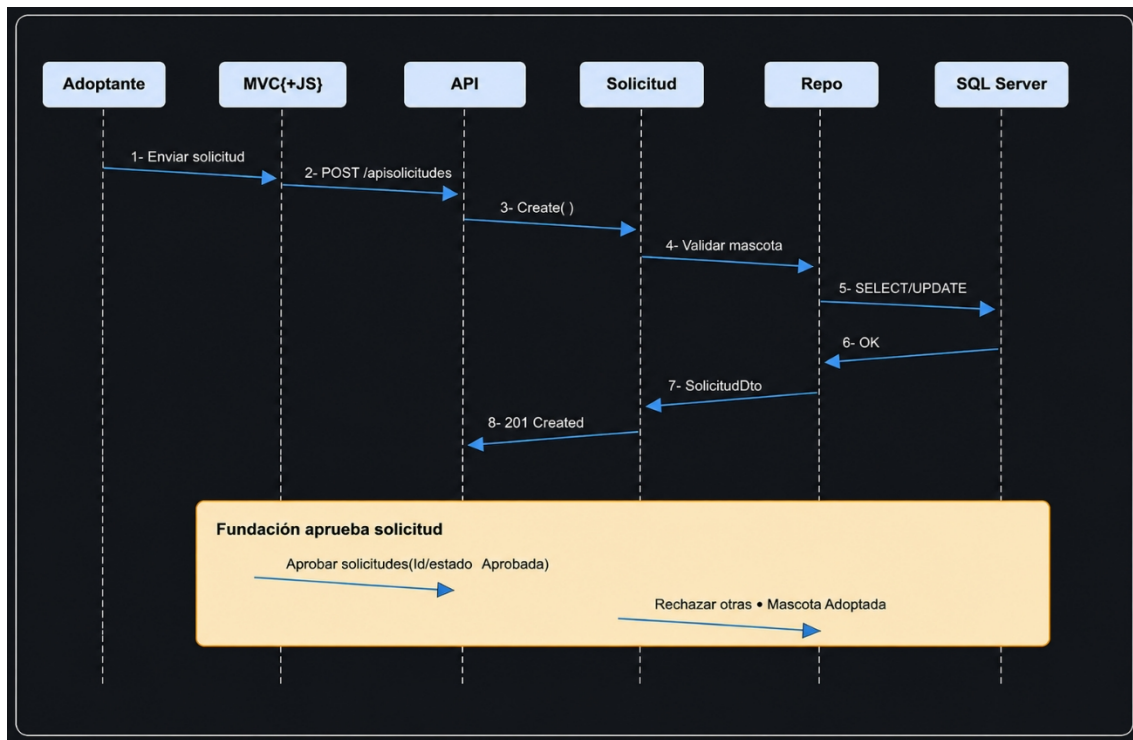
10. Diagrama de casos de uso



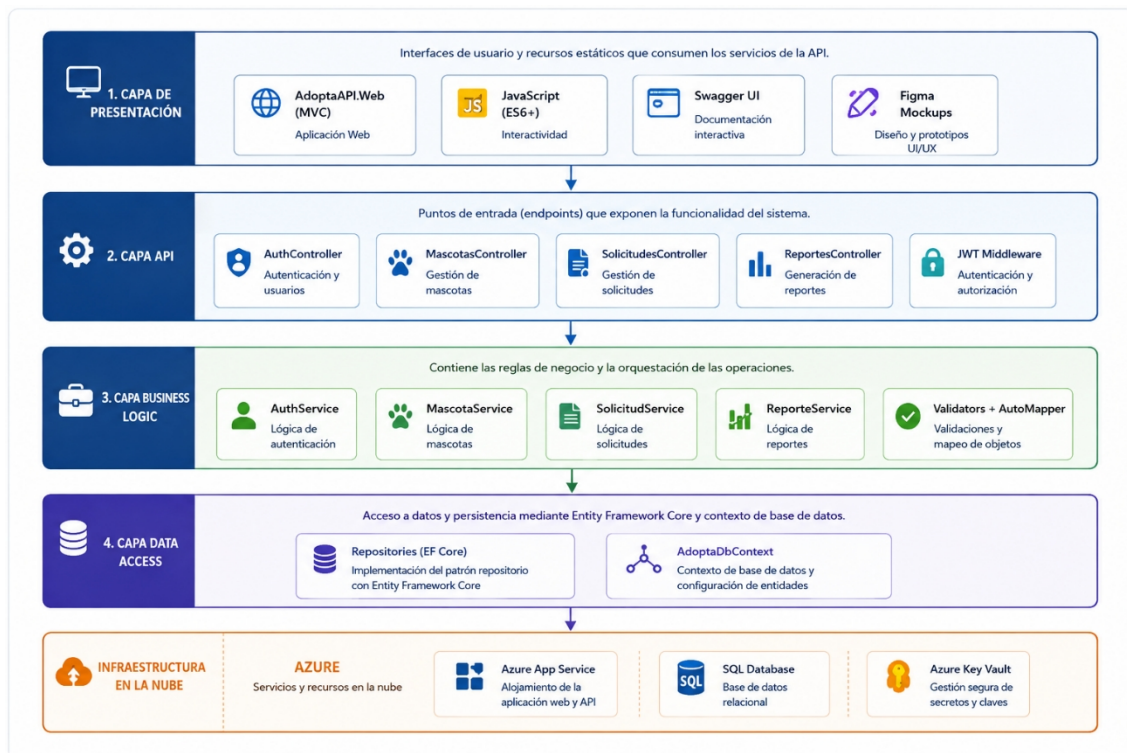
11. Diagrama de clases



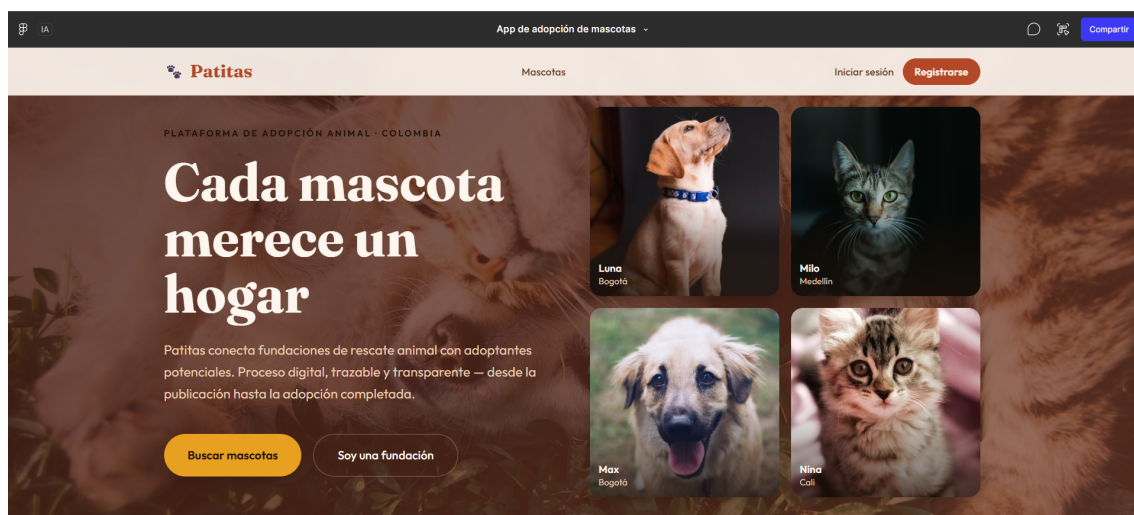
12. Diagrama de secuencia



13. Diagrama de componentes



<https://www.figma.com/make/xCD5ROT9ZUGXGvQNgrjYjW/App-de-adopci%C3%B3n-de-mascotas?fullscreen=1&t=XRuRs1CrffpTfesV-1&code-node-id=0-6>



AdoptaAPI — Informe de Proyecto v1.0 · Agosto 2026

Samanta Rojas · Killiam García · Jonnathan Rey · Santiago

Cuervo ASP.NET Core MVC 8 · SQL Server · Azure ·

Agile/Scrum